

练习题（十五）

课程：编译原理 | 主题：属性文法、中间代码生成（回填技术与四元式）

一、单选题

1. 令非终结符 B 表示布尔表达式。在布尔表达式的回填技术中，为非终结符 B 设置两个综合属性：
 $B.truelist$ 指向一个包含跳转指令的列表，这些指令最终获得的目标标号就是当 B 为真时控制流应该转向的指令的标号； $B.falselist$ 指向一个包含跳转指令的列表，这些指令最终获得的目标标号就是当 B 为假时控制流应该转向的指令的标号。则下列 SDT 中不正确的是（ ）。【布尔表达式的回填】

- A. $B \rightarrow E_1 \text{ relop } E_2 \quad \{ B.truelist = \text{makelist}(\text{nextquad}); B.falselist = \text{makelist}(\text{nextquad}+1); \text{gen}('goto _'); \}$
- B. $B \rightarrow \text{true} \quad \{ B.truelist = \text{makelist}(\text{nextquad}); \text{gen}('goto _'); \}$
- C. $B \rightarrow \text{false} \quad \{ B.falselist = \text{makelist}(\text{nextquad}); \text{gen}('goto _'); \}$
- D. $B \rightarrow (B_1) \quad \{ B.truelist = B_1.truelist; B.falselist = B_1.falselist; \}$

2. 在下面的语句中，（ ）不需要回填技术。

- A. 赋值语句
- B. goto 语句
- C. 条件语句
- D. 循环语句

3. 四元式之间的联系是通过（ ）实现的。【四元式】

- A. 指示器
- B. 临时变量
- C. 符号表
- D. 程序变量

4. 四元式表示法的优点为（ ）。【四元式】

- A. 不便于优化处理, 但便于表的更动
- B. 不便于优化处理, 但节省存储空间
- C. 便于优化处理, 也便于表的更动
- D. 便于表的更动, 也节省存储空间

二、判断题

5. 在回填技术中，生成一个跳转指令时，暂时不指定该跳转指令的目标标号。（ ）【回填】

6. 在回填技术中，同一个列表 list 中的跳转指令具有相同的目标标号。（ ）【回填】

7. 在回填技术中，同一个列表 list 中的跳转指令可能具有不同的目标标号。（ ）【回填】

8. 在回填技术中，等到能够确定正确的目标标号时，才去填充指令的目标标号。（ ）【回填】

三、参考答案

题号	1	2	3	4	5	6	7	8
答案	A	A	B	C	√	√	×	√

四、答案解析

1. 【答案】A

对于布尔表达式的条件跳转 $B \rightarrow E_1 \text{ relop } E_2$ ，语义上需要生成两条跳转指令：一条是条件满足时跳转（对应真），另一条是无条件跳转（对应假）。标准的中间代码生成序列应占用两个 quad 位置：

1) if E1.addr relop E2.addr goto _（绑定为真，占 nextquad）

2) goto _（绑定为假，占 nextquad + 1）

选项 A 中只显式调用了一次 `gen('goto _')`，且没有生成对应的条件跳转代码，其语义不完整，故错误。B、C、D 项均符合回填 SDT 的规范。

2. 【答案】A

回填技术主要解决控制流翻译中**向前跳转**时目标标号未确定的问题。条件语句（if-else）和循环语句（while、for）均涉及大量前向条件或无条件跳转；goto 语句在跳转到未定义的标签时也依赖回填。而**赋值语句**属于纯顺序执行结构，无控制流分支或跳转，因此完全不需要回填技术。

3. 【答案】B

在中间代码中，四元式的一般结构为 (op, arg1, arg2, result)。由于结果项 result 是被显式命名并写出的**临时变量**或程序变量，当一条四元式需要引用前一条四元式的结果时，它们是通过**共用同一个临时变量名**来建立逻辑和数据流上联系的。（注：通过指示器或行号隐式建立联系的是三元式表示法）。

4. 【答案】 C

四元式显式包含结果项，这带来了两大优势：**一是便于优化处理**，在进行死代码消除或循环代码外提时，由于参数传递不依赖物理行号，移动四元式位置不会破坏整体引用结构；**二是便于表的更动**，在中间任意插入或删除一条四元式时，不需要像三元式那样去成片修改其他指令的操作数。

5. 【答案】 √

正确。这正是回填（Backpatching）方法的核心机制：当遇到跳转指令且其目标行号在当前尚未被翻译出来时，先生成一条缺失目标地址的跳转空壳指令（如 `goto _`），留待后填。

6 & 7. 【答案】 6题 √；7题 ×

回填链表（如 `truelist`、`falselist`）的基本设计目的，就是将**所有跳转到同一个目标地址**的待补全指令收集在同一个链表中。当目标行号明确时，调用 `backpatch(list, target)` 将表中所有指令的目标统一回填为该标号。因此，同一个 `list` 中的跳转指令必然具有相同的目标标号。

8. 【答案】 √

正确。回填技术采用“先生成、后补全”的推迟绑定策略。只有在语法分析流推进到能够完全确定跳转目标的正确行号（即 `nextquad`）时，才会逆向回修，将正确的标号填入之前的留空指令中。
